

Searching

Introduction

Search algorithms are fundamental techniques in computer science that involve looking for a specific item, value, or piece of information within a dataset.

The concept of computer search parallels tasks we perform daily, such as finding a book in a library, locating a contact in a phone directory, or even searching for a specific word in a document.

Here, we will cover two types of search algorithms:

- Linear Search
- Binary Search

First, we will learn about the linear search algorithm.

Introduction

The linear search algorithm is used to find an element within a vector.

In this algorithm, we sequentially check each element of the vector until the desired element is found.

Suppose we have the following vector, and we have to find the index of element **4** (the target value).

	9	10	5	8	7	4	11	6	15	3
index	0	1	2	3	4	5	6	7	8	9

Figure: Introduction to Linear Search

In linear search, we sequentially check each index to see if the target value (in this case, element **4**) is present.

We start from **index 0**, then **index 1**, **index 2**, and so on, until the element is found. In this particular case, the target value is at **index 5**.

Once this element is found, the linear search ends.

Unsuccessful Search

Suppose we have to find the index of element **20** (the target value) in the given vector.

	9	10	5	8	7	4	11	6	15	3
index	0	1	2	3	4	5	6	7	8	9

Figure: Unsuccessful Search

In this case, the linear search algorithm checks each element one by one.

The element 20 is not present in the vector, so the algorithm ends after reaching the end of the vector. This is called an **unsuccessful search**.

Thought Process to Implement Linear Search

Now that we know how linear search works, let's see how we can implement it in C++.

1. Access all vector elements

In linear search, we need to compare the target value with every element in a vector.

To access every vector elements, we can use a for loop that runs up to the end of the vector:

```
for (int i = 0; i < arr.size(); ++i) {  
}
```

2. Compare target value with vector elements

If the target element matches with any vector element, we return the index of that element.

```
for (int i = 0; i < arr.size(); ++i) {  
    if (arr[i] == target)  
        return i;  
}
```

3. Hand unsuccessful search.

If the target element doesn't exist in the vector, we return **-1** to signify an unsuccessful search.

```
for (int i = 0; i < arr.size(); ++i) {  
    if (arr[i] == target)  
        return i;  
}  
  
return -1;
```

Next, we will create a working program of linear search.

Source Code: Linear Search

```
1 // Template to perform linear search on a vector.
2 template<typename T>
3 int linear_search(const vector<T>& arr, T target) {
4
5     // Iterate through all elements in the vector.
6     for(int i = 0; i < arr.size(); ++i) {
7         // Check if the current element matches the target.
8         if(arr[i] == target)
9             // Return the index of the element if found.
10            return i;
11     }
12     // Return -1 if the target is not found.
13     return -1;
14 }
```

[Run Code >>](#)

Output

Element 4 is found at index 5

Was this helpful?

Yes 

No 

Find the Occurrence of a Number

Easy



New to Practice?



Take a tour

Create a program to find the occurrence of a given number in a vector.

- Create a function named `count_occurrences()` that takes two arguments: `arr` (a vector) and `n` (an integer whose occurrences we need to find).
- Find the count of occurrences of `n` in `arr` and return this count.
- Print the result from outside the function.

Note: We will use a predefined vector, `{3, 3, 4, 5, 6, 6, 6, 2}`, for this problem.

Example

Test Input

6

Expected Output

3

Reason: The occurrence of the value **6** in this vector is **3** because it is present in **3** places.

Was this helpful?

Yes

No

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  // Template to count occurrences of a target value within a vector.
7  template<typename T>
8  int count_occurrences(const vector<T>& arr, T target) {
9      // Initialize count to zero.
10     int count = 0;
11
12     // Loop through each element in the vector.
13     for(const auto elem: arr) {
14         // If the element equals the target,
15         // increment the count.
16         if(elem == target)
17             ++count;
18     }
19     // Return the total count of occurrences.
20     return count;
21 }
22
23 int main() {
24     // Initialize a vector of integers.
25     // And set the target to search for.
26     vector<int> nums {3, 3, 4, 5, 6, 6, 6, 2};
27     int target = 6;
28
29     // Call count_occurrences and output the result.
30     cout << count_occurrences(nums, target) << '\n';
31     return 0;
32 }
```

main.cpp

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  // Template to count occurrences of a target value within a vector.
7  template<typename T>
8  int count_occurrences(const vector<T>& arr, T target) {
9      // Initialize count to zero.
10     int count = 0;
11
12     // Loop through each element in the vector.
13     for(const auto elem: arr) {
14         // If the element equals the target,
15         // increment the count.
```

Output 1

🎉 Great! You passed all the test cases!

Status	Cases	Expected Output	Your Output
Passed	Test 1	3	3

Complexity Analysis

The time and space complexities for linear search is given by the table below:

Best Case Time Complexity	$O(1)$
Worst Case Time Complexity	$O(n)$
Average Time Complexity	$O(n)$
Space Complexity	$O(1)$

Introduction

Similar to linear search, binary search is used to find an element within a vector.

However, binary search can **only be implemented in a sorted vector**. If vector elements are unsorted, we need to sort them first.

Binary search works by repeatedly dividing the vector in half until the possible location of the item is narrowed down to just one.

Let's understand this with the help of images.

Working of Binary Search

Suppose we have the following sorted vector, and we have to find the index of element **4** (the target value).

	3	4	5	6	7	8	9
index	0	1	2	3	4	5	6

Figure: A Sorted vector

1. Divide the vector into two parts.

Consider, `low` is the index of the first element of the scope and `high` is the index of the last element of the scope.

	3	4	5	6	7	8	9
index	0	1	2	3	4	5	6
	↑						↑
	low						high

Figure: Set Low and High

Now we need to divide our vector into two equal halves. So, we split the vector over the middle element.



Figure: Find the mid Index

- If the mid element is the target element, we have found our target.
- If the target is smaller than the mid element, we narrow the scope of the target element to the left side of the mid element.
- If the target is greater than the mid element, we narrow the scope of the target to the right side of the mid element.

We repeat this process until we find our target element or the length of the search scope falls down to zero.

2. Repeat for the left side.

Since our target element **4** is less than the middle element **6**, we repeat the steps on the lower halves (elements to the left of the middle element).

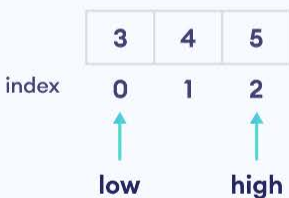


Figure: Update the high Index

Again, we divide the vector into two halves and compare our target element with the middle element.

This time, the target element is equal to the middle element. Hence, our search is completed.

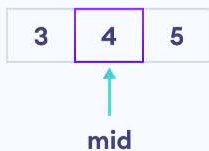


Figure: Found the Target Element

Quiz: Why can't we use binary search on an unsorted vector?

A: Because binary search relies on dividing the vector.

B: Because binary search requires repeatedly comparing the middle element with the target.

C: Because the vector has too many elements.

D: Because it's less efficient.

Quiz: Why can't we use binary search on an unsorted vector?

A: Because binary search relies on dividing the vector.

B: Because binary search requires repeatedly comparing the middle element with the target.

Correct Answer!



Binary search repeatedly compares the middle element with the target and uses the sorted nature of the vector to eliminate half of the remaining elements.

C: Because the vector has too many elements.

D: Because it's less efficient.

Thought Process: Binary Search

Now, let's see how we can implement binary search.

We'll create a function named `binary_search()` that takes a vector `arr` as its parameter.

1. Assign the high and low index.

Initially, our search scope is the entire vector, so we initialize the `low` index as the first element and the `high` index as the last element of the vector.

```
int low = 0; // Starting index of the search range
int high = arr.size() - 1; // Ending index of the search range
```

2. Divide the search range over the middle index.

To divide the vector into two equal halves, we have to find the middle index.

```
int mid = low + (high - low) / 2;
```

Now we will compare the target element with the element at the `mid` index.

Thought Process: Binary Search

3. Compare middle element with target value.

Comparison 1: Target value equal to the middle element

If the target value is equal to the middle element, we have found the element and our search is completed.

We can return the index of the middle element.

```
if(target == arr[mid])  
    return mid;
```

Comparison 2: Target value less than the middle element

If the target value is less than the middle element, we continue the search on the left side of the middle element.

In this case, we change the value of the `high` index to `mid - 1`.

```
if(target < arr[mid])  
    high = mid - 1;
```

Comparison 3: Target value greater than the middle element

If the target value is greater than the middle element, we continue the search on the right side of the middle element.

In this case, we change the value of the `low` index to `mid + 1`.

```
if (target > arr[mid])  
    low = mid + 1;
```

At this point, we either find the target element or update the low/high index to reduce our search scope for the next iteration.

Thought Process: Binary Search

4. Repeat until we get the value or run out of elements.

After updating the high or low index, we have to repeat the step of dividing the vector and comparing the middle with the target value.

Hence, we can use a while loop that runs until the low index is greater than the high index.

```
while (low <= high) {  
  
}
```

If low is greater than high, we can say our target value is not in the vector, and the search is unsuccessful.

Let's now create a working program for binary search next.

Source Code: Binary Search

```
1  template<typename T>
2  int binary_search(const vector<T>& arr, T target) {
3      int low = 0; // Starting index of the search range
4      int high = arr.size() - 1; // Ending index of the search range
5
6      while(low <= high) {
7          // Calculate the middle index to divide the search range.
8          int mid = low + (high - low) / 2;
9
10         // Check if the target is present at mid.
11         if(target == arr[mid])
12             return mid;
13
14         // If target is smaller than mid,
15         // reduce the search range to the left half.
16         else if(target < arr[mid])
17             high = mid - 1;
18
19         // If target is larger,
20         // adjust the search range to the right half.
21         else
22             low = mid + 1;
23     }
24     // Target was not found in the array.
25     return -1;
26 }
```

[Run Code >>](#)

Output

Element 7 is found at index 3



Tip: We encourage you to add `print()` statements in the above program to display `low`, `high` and `mid` values to get an idea of how binary search updates these values.

Binary Search

Medium



New to Practice?



Take a tour

Program Description

Can you write the binary search program on your own?

- Create a function named `binary_search()` that takes two arguments: `arr` (a vector) and `target` (an integer).
- Use binary search to find the index of `target` in the `arr` vector.
- Return the index of `target` if the element is present in the vector. If not, return `-1`.
- Print the returned value from outside the function

Note: We will use a predefined sorted vector, `{4, 5, 9, 12, 55, 79, 88, 99}` for this practice.

Example

Test Input

4

Expected Output

0

Was this helpful?

Yes

No

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  // Template to perform binary search on a sorted array.
7  template<typename T>
8  int binary_search(const vector<T>& arr, T target) {
9      int low = 0; // Starting index of the search range
10     int high = arr.size() - 1; // Ending index of the search range
11
12     while(low <= high) {
13         // Calculate the middle index to divide the search range.
14         int mid = low + (high - low) / 2;
15
16         // Check if the target is present at mid.
17         if(target == arr[mid])
18             return mid;
19
20         // If target is smaller than mid,
21         // reduce the search range to the left half.
22         else if(target < arr[mid])
23             high = mid - 1;
24
25         // If target is larger,
26         // adjust the search range to the right half.
27         else
28             low = mid + 1;
29     }
30     // Target was not found in the array.
31     return -1;
32 }
33
34 int main() {
35     // Initialize a vector of integers and specify the target value.
36     vector<int> nums {4, 5, 9, 12, 55, 79, 88, 99};
37     int target = 4;
38     // Call binary_search function and capture the index result.
39     int idx = binary_search(nums, target);
40
41     // Output the result of the search.
42     cout << idx << '\n';
43     return 0;
44 }
```

Source Code: Binary Search Using Recursion

We can also perform binary search using recursion.

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  // Template to perform binary search on a sorted vector using recursion
7  template <typename T>
8  int binary_search(const vector<T>& arr, T target, int low, int high) {
9      if(high >= low) {
10
11          // Calculate middle index to divide the vector.
12          int mid = low + (high - low) / 2;
13
14          // Check if the target is present at mid.
15          if(target == arr[mid])
16              return mid;
17
18          // If target is smaller than mid,
19          // then it can only be present in left subarray.
20          else if(target < arr[mid])
21              return binary_search(arr, target, low, mid - 1);
22
23          // Else the target can only be in right subarray.
24          else
25              return binary_search(arr, target, mid + 1, high);
26      }
27      // Target was not found in the vector.
28      return -1;
29  }
30
31  int main() {
32      // Search for element 5 in a vector of integers.
33      vector<int> nums {3, 4, 5, 6, 7, 8, 9};
34      int target = 5;
35
36      // Initiate binary search
37      int idx = binary_search(nums, target, 0, nums.size() - 1);
38
39      // Output the result of the search.
40      if(idx == -1)
41          cout << "Element " << target << " is not found in the vector." << endl;
42      else
43          cout << "Element " << target << " is found at index " << idx << endl;
44      return 0;
45  }
```

Run Code >>

Sample Output

Element 5 is found at index 2

Here, we have used recursion to perform the binary search.

However, the logic behind this program is similar to how we performed binary search using the iterative process. We will discuss it next.

Explanation: Binary Search Using Recursion

We call the `binary_search()` function recursively until `low` is greater than `high` (similar to the iterative process).

```
if(high >= low) {  
    // 1. update the mid element  
  
    // 2. if target is equal to mid, return index of mid  
  
    // 3. update either low or high and make recursive calls  
}  
return -1;
```

If `low` is greater than `high`, it means the target element is not in the vector. We return `-1` in this case.

1. Update the mid element.

```
// Calculate middle index to divide the vector.  
int mid = low + (high - low) / 2;
```

2. If the target is equal to mid, return the index of mid.

```
// Check if the target is present at mid.  
if(target == arr[mid])  
    return mid;
```

If the middle element is equal to the target, we have found the element. In this case, we return the index of the middle element, and the recursion ends.

3. Update either low or high elements and make recursive calls.

Similar to the iterative process:

- If the `target` element is less than the `mid` element, we update `high` to `mid - 1`.
- If the `target` element is greater than the `mid` element, we update `low` to `mid + 1`.

Then we call the `binary_search()` function with these updated values.

```
// If target is smaller than mid,  
// then it can only be present in left subarray.  
else if(target < arr[mid])  
    return binary_search(arr, target, low, mid - 1);  
  
// Else the target can only be in right subarray.  
else  
    return binary_search(arr, target, mid + 1, high);
```

Was this helpful?

Yes 

No 

Binary Search Using Recursion

Medium



New to Practice?



Take a tour

Program Description

Can you write the binary search program on your own?

- Create a function named `binary_search()` that takes two arguments: `arr` (a vector) and `target` (an integer).
- Use recursive binary search to find the index of `target` in the `arr` vector.
- Return the index of `target` if the element is present in the vector. If not, return `-1`.
- Print the returned value from outside the function

Note: We will use a predefined sorted vector, `{4, 5, 9, 12, 55, 79, 88, 99}` for this practice.

Example

Test Input

4

Expected Output

0

Was this helpful?

Yes

No

Complexity Analysis

The time and space complexities of binary search is given below:

Best Case Time Complexity	$O(1)$
Worst Case Time Complexity	$O(\log n)$
Average Time Complexity	$O(\log n)$
Space Complexity	$O(1)$

To learn more about the complexity and applications of binary search, visit [this blog](#).

Was this helpful?

Yes 

No 

How to Write Recurrence Relation?

Zohaib Ali

Example # 1

```
void Test(int n)
{
    if (n > 0)
    {
        printf("%d", n);
        Test(n-1);
    }
}
```

Solution

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

$T(n)$ — `void Test(int n)`
 {
 if (n > 0)
 {
 1 — `printf("%d", n);`
 $T(n-1)$ — `Test(n-1);`
 }
 }

 $T(n) = T(n-1) + 1$

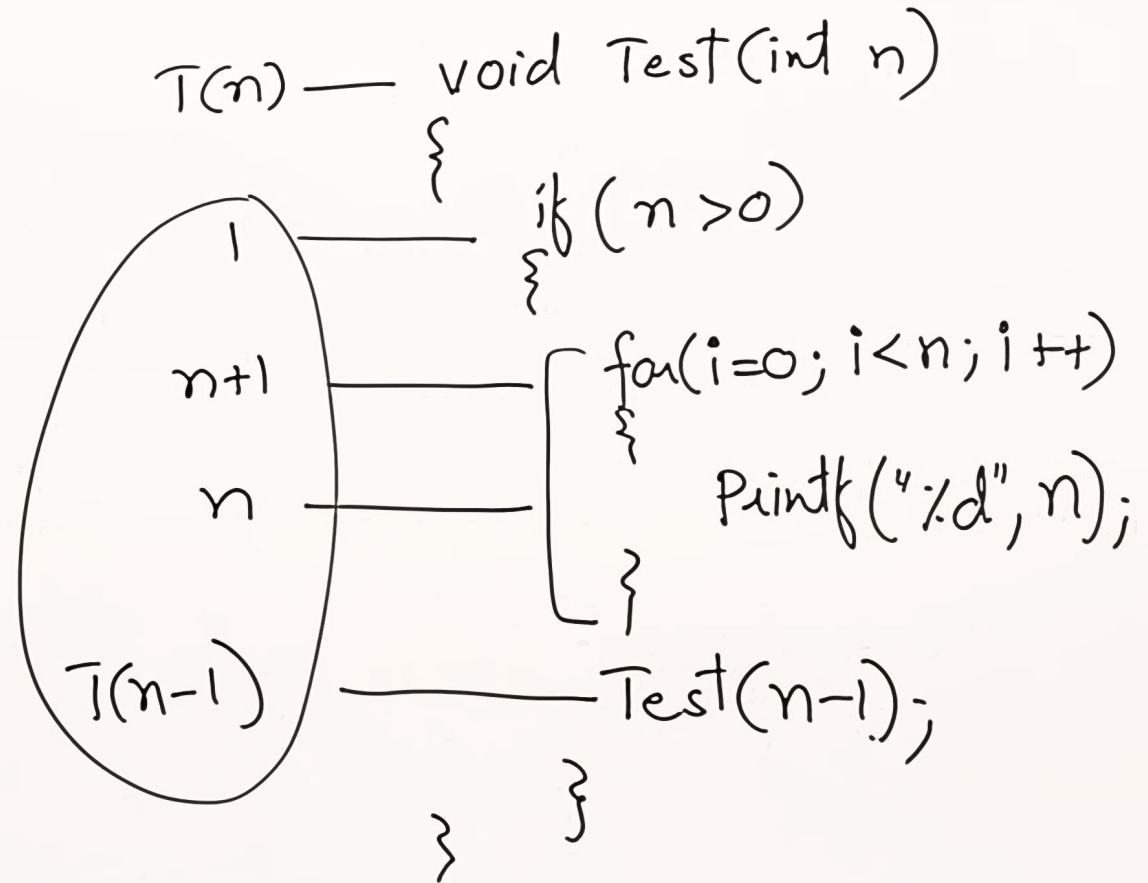
Example # 2

```
void Test(int n)
{
    if (n > 0)
    {
        for (i = 0; i < n; i++)
        {
            printf("%d", n);
        }
        Test(n-1);
    }
}
```

Solution

$$T(n) = T(n-1) + n$$

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + n & n>0 \end{cases}$$



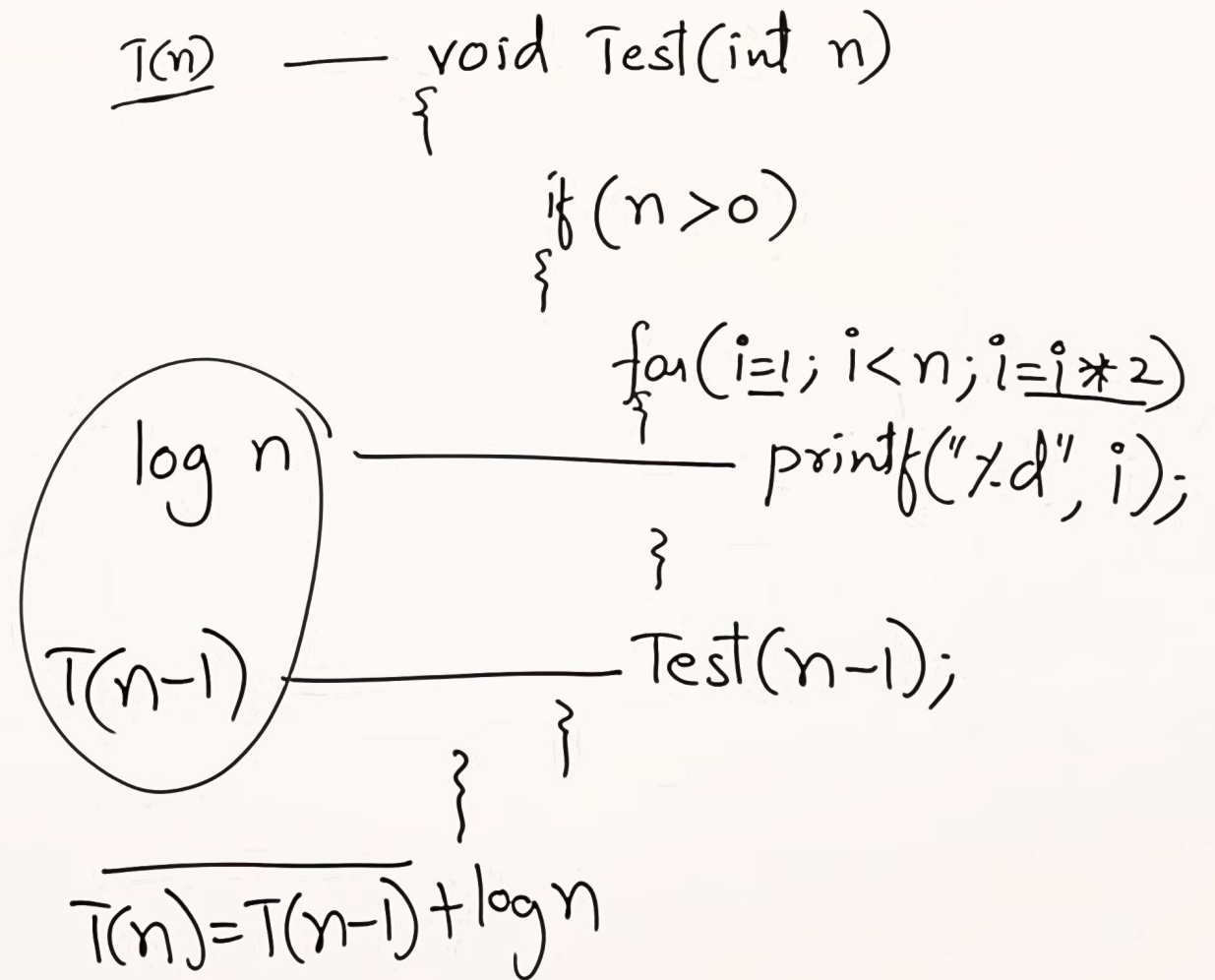
$$T(n) = T(n-1) + 2n + 2$$

Example # 3

```
void Test(int n)
{
    if (n > 0)
    {
        for (i = 1; i < n; i = i * 2)
        {
            printf("%d", i);
        }
        Test(n - 1);
    }
}
```

Solution

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + \log n & n>0 \end{cases}$$

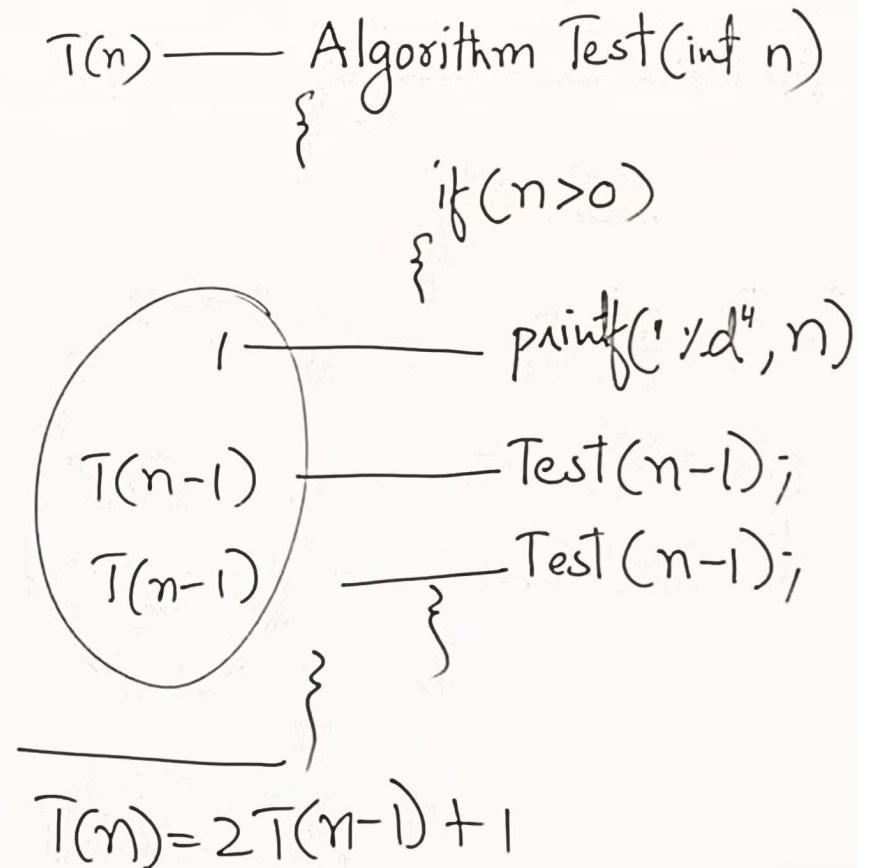


Example # 3

```
Algorithm Test(int n)
{
    if(n > 0)
    {
        printf("%d", n);
        Test(n-1);
        Test(n-1);
    }
}
```

Solution

$$T(n) = \begin{cases} 1 & n=0 \\ 2T(n-1) + 1 & n>0 \end{cases}$$

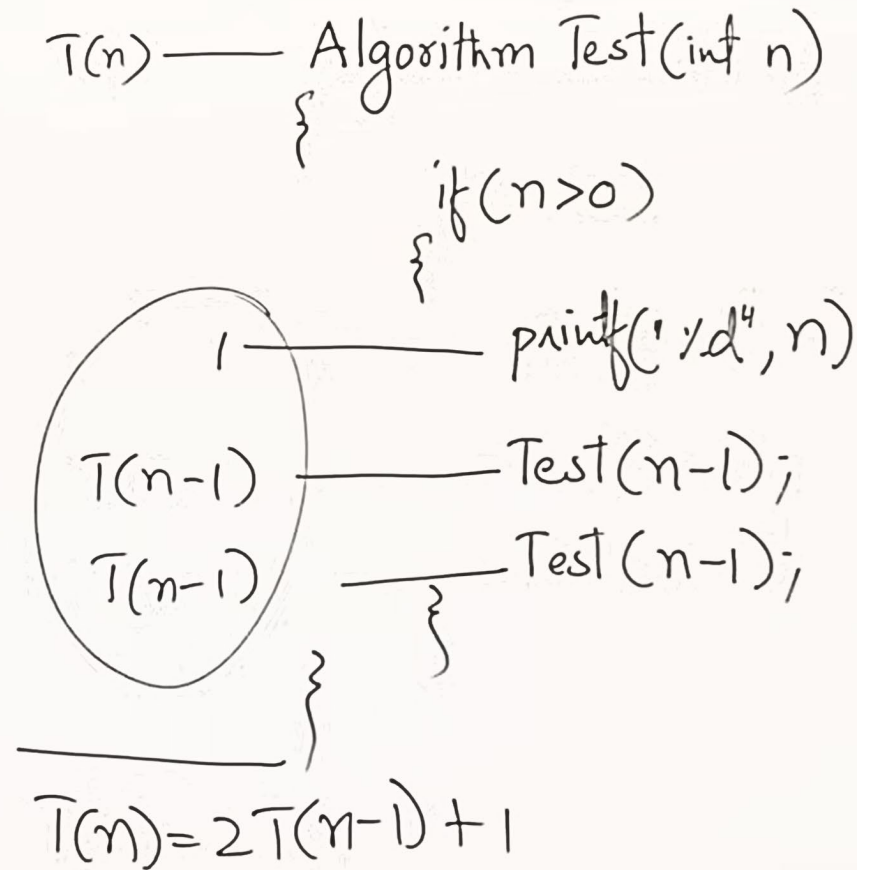


Example # 4

```
Algorithm Test(int n)
{
    if (n > 0)
    {
        printf("%d", n);
        Test(n-1);
        Test(n-1);
    }
}
```

Solution

$$T(n) = \begin{cases} 1 & n=0 \\ 2T(n-1) + 1 & n>0 \end{cases}$$



Master Theorem

Recurrence Relation

Master theorem for Decreasing Functions

$$T(n) = aT(n-b) + f(n)$$

$a > 0$ $b > 0$ and $f(n) = O(n^k)$ where $k \geq 0$

Case

1. if $a < 1$

$$\begin{matrix} O(n^k) \\ O(f(n)) \end{matrix}$$

$$T(n) = T(n-1) + 1 \rightarrow O(n)$$

$$T(n) = T(n-1) + n \rightarrow O(n^2)$$

$$T(n) = T(n-1) + \log n \rightarrow O(n \log n)$$

2. if $a = 1$

$$\begin{matrix} O(n^{k+1}) \\ O(n * f(n)) \end{matrix}$$

3. if $a > 1$

$$O(n^k a^{n/b})$$

$$T(n) = 2T(n-2) + 1 \rightarrow O(2^{n/2})$$

$$T(n) = 3T(n-1) + 1 \rightarrow O(3^n)$$

$$T(n) = 2T(n-1) + n \rightarrow O(n2^n)$$

Master Theorem for Dividing Functions

$$T(n) = aT(n/b) + f(n)$$

$$\begin{matrix} a \geq 1 \\ b > 1 \end{matrix} \quad f(n) = O(n^k \log^p n)$$

① $\log_b a$

② k

Case 1: if $\log_b a > k$ then $O(n^{\log_b a})$

Case 2: if $\log_b a = k$

if $p > -1$ $O(n^k \log^{p+1} n)$

if $p = -1$ $O(n^k \log \log n)$

if $p < -1$ $O(n^k)$

Case 3: if $\log_b a < k$

if $p \geq 0$ $O(n^k \log^p n)$

if $p < 0$ $O(n^k)$

Introduction to Master Theorem

The master theorem is used to find the time complexity of recursive functions.

If you don't exactly know how recursive functions work, we recommend you take the [Python Recursion](#) course before you proceed.

We can use the master theorem for primarily two types of recursive function:

- Dividing Function
- Decreasing Function

Was this helpful?

Yes 

No 

Quiz: What is the master theorem used for?

Tap an option below to answer:

A: To find the time complexity of recursive functions.

B: To find the time complexity of any function.

C: To find the time complexity of nested loops.

D: None of the above.

Quiz: What is the master theorem used for?

A: To find the time complexity of recursive functions.



Correct Answer!

The master theorem is a simple and direct way of finding the time complexity of recursive functions.

B: To find the time complexity of any function.

C: To find the time complexity of nested loops.

D: None of the above.

Recurrence Relation for Dividing Function

Mathematically speaking, the master theorem provides a solution to the **recurrence relation**. Let's learn how we can find the recurrence relation for a dividing function.

Suppose we have a program like this.

```
def test(n):
    if n > 0:
        print(n)
        test(n // 2)
```

If the `test()` function takes `T(n)` time to execute, then

```
def test(n): ----- T(n)
    if n > 0: ----- 1
        print(n) ----- 1
        test(n // 2) ----- T(n / 2) unit of time
```

Figure: Recurrence Relation for Dividing Function

From the image, we can say

$$T(n) = T\left(\frac{n}{2}\right) + 1 + 1$$

$$= T\left(\frac{n}{2}\right) + 2$$

In place of constant **2**, we can write **1** or simply represent it with `c`.

$$T(n) = T\left(\frac{n}{2}\right) + 1$$



Note: We have been using `f(n)` to represent a function. However, for recurrence relationships, we use `T(n)` to represent a function.

Solving Recurrence Relation with Master Theorem

To solve an equation using recurrence relation for a dividing function, the function must be in the following format:

$$T(n) = \frac{a}{b} + f(n)$$

where,

$$f(n) = \theta(n^k (\log n)^p),$$

$$a \geq 1, b > 1,$$

$$k \geq 1,$$

$$p = \text{real number}$$

Now, let's observe the equation for three different cases:

- **Case 1:** $\log_b a > k$
- **Case 2:** $\log_b a = k$
- **Case 3:** $\log_b a < k$

Let's explore each of these cases next.

$\log_b a > k$

If $\log_b a > k$, then $T(n) = \Theta(n^{\log_b a})$

Was this helpful?

Yes 

No 

Dividing Function

Let's first understand what a dividing function is. Then, we will learn to calculate the time complexity of dividing functions using the master theorem.

Here's an example of a dividing function:

```
1 def test(n):  
2     if n > 0:  
3         print(n)  
4         test(n // 2)  
5  
6 test(20)
```

Run Code >>

Output

```
20  
10  
5  
2  
1
```

Here, `test()` is a recursive function because we have called the same `test()` function from inside it.

In each recursive call, the argument is divided by **2** using the `test(n // 2)` statement. Hence, `test()` is a dividing function (divide and conquer).

$$\log_b a = k$$

If $\log_b a = k$, then we should check values of p for three cases:

1. if $p > -1$, then $T(n) = \Theta(n^k \log^{p+1} n)$
2. if $p = -1$, then $T(n) = \Theta(n^k \log \log n)$
3. if $p < -1$, then $T(n) = \Theta(n^k)$

Was this helpful?

Yes 

No 

$\log_b a < k$

If $\log_b a < k$, then we should check values of p for two cases:

1. if $p \geq 0$, then $T(n) = \Theta(n^k \log^p n)$
2. if $p < 0$, then $T(n) = \Theta(n^k)$

Was this helpful?

Yes 

No 

Example 1

The recurrence relation of the dividing function we previously wrote is:

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Now let's calculate its time complexity using the master theorem.

Step 1: Comparing the recurrence relation with the general form.

The general form of $T(n)$ is:

$$T(n) = aT\left(\frac{n}{b}\right) + \theta(n^k(\log n)^p)$$

Step 2: Modifying recurrence relation to match the general form.

$$T(n) = 1 * T\left(\frac{n}{2}\right) + \theta(n^0(\log n)^0)$$

Step 3: Finding a , b , k and p and checking if the master theorem is possible.

$$\begin{aligned}a &= 1 \\b &= 2 \\k &= 0 \\p &= 0\end{aligned}$$

Since $a \geq 1$, $b > 1$, $k \geq 0$, and p is a real number, the master theorem is possible.

Step 4: Comparing $\log_b a$ and k and applying the matching rule.

Here,

$$\begin{aligned}\log_b a &= \log_2 1 = 0 \\k &= 0\end{aligned}$$

Since $\log_b a = k$ and $p > -1$, let's apply the formula to find the time complexity for this scenario.

$$\begin{aligned}T(n) &= \theta(n^k \log^{p+1} n) \\&= \theta(n^0 \log^{0+1} n) \\&= \theta(1 * \log^1 n) \\&= \theta(\log n)\end{aligned}$$

Example 2

Suppose we have the following recurrence relation:

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Now let's calculate its time complexity using the master theorem.

Step 1: Comparing the recurrence relation with the general form.

The general form of $T(n)$ is:

$$T(n) = aT\left(\frac{n}{b}\right) + \theta(n^k(\log n)^p)$$

Step 2: Modifying recurrence relation to match the general form.

$$T(n) = 4T\left(\frac{n}{2}\right) + \theta(n^1(\log n)^0)$$

Step 3: Finding a , b , k and p and checking if the master theorem is possible

$$\begin{aligned}a &= 4 \\b &= 2 \\k &= 1 \\p &= 0\end{aligned}$$

Since $a \geq 1$, $b > 1$, $k \geq 0$, and p is a real number, the master theorem is possible.

Step 4: Comparing $\log_b a$ and k and applying the matching rule.

Here,

$$\begin{aligned}\log_b a &= \log_2 4 = 2 \\k &= 1\end{aligned}$$

Since $\log_b a > k$, let's apply the formula to find the time complexity for this scenario.

$$\begin{aligned}T(n) &= \theta(n^{\log_b a}) \\&= \theta(n^2)\end{aligned}$$

Example 3

Suppose we have the following recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

Now let's calculate its time complexity using the master theorem.

Step 1: Comparing the recurrence relation with the general form.

The general form of $T(n)$ is:

$$T(n) = aT\left(\frac{n}{b}\right) + \theta(n^k(\log n)^p)$$

Step 2: Modifying recurrence relation to match the general form.

$$T(n) = 2 * T\left(\frac{n}{2}\right) + \theta(n^2(\log n)^0)$$

Step 3: Finding a , b , k , and p and checking if the master theorem is possible.

$$\begin{aligned}a &= 2 \\b &= 2 \\k &= 2 \\p &= 0\end{aligned}$$

Since $a \geq 1$, $b > 1$, $k \geq 0$, and p is a real number, the master theorem is possible.

Step 4: Comparing $\log_b a$ and k and applying the matching rule.

Here,

$$\begin{aligned}\log_b a &= \log_2 2 = 1 \\k &= 2\end{aligned}$$

Since $\log_b a$ and $p \geq 0$, let's apply the formula to find the time complexity for this scenario.

$$\begin{aligned}T(n) &= \theta(n^k(\log n)^p) \\&= \theta(n^2(\log n)^0) \\&= \theta(n^2 * 1) \\&= \theta(n^2)\end{aligned}$$

Example 4

Suppose we have the following recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + \theta(n^2 \log^2 n)$$

Now let's calculate its time complexity using the master theorem.

Step 1: Comparing the recurrence relation with the general form.

The general form of $T(n)$ is:

$$T(n) = aT\left(\frac{n}{b}\right) + \theta(n^k (\log n)^p)$$

Step 2: Modifying recurrence relation to match the general form.

$$T(n) = 2T\left(\frac{n}{2}\right) + \theta(n^2 (\log n)^2)$$

Step 3: Finding a , b , k , and p and checking if the master theorem is possible.

$$\begin{aligned}a &= 2 \\b &= 2 \\k &= 2 \\p &= 2\end{aligned}$$

Since $a \geq 1$, $b > 1$, $k \geq 0$, and p is a real number, the master theorem is possible.

Step 4: Comparing $\log_b a$ and k and applying the matching rule.

Here,

$$\begin{aligned}\log_b a &= \log_2 2 = 1 \\k &= 2\end{aligned}$$

Since $\log_b a < k$ and $p \geq 0$, let's apply the formula to find the time complexity for this scenario.

$$\begin{aligned}T(n) &= \theta(n^k (\log n)^p) \\&= \theta(n^2 (\log n)^2) \\&= \theta(n^2 \log^2 n)\end{aligned}$$



Remember: It is not necessary to remember all the cases and their respective formula for the master theorem. You can always look it up whenever you need it.

Quiz: Which case should be considered when finding the time complexity of the following recurrence relation?

$$T(n) = 16T\left(\frac{n}{4}\right) + n$$

A: $\log_b a > k$

B: $\log_b a = k$

C: $\log_b a < k$

D: $\log_b a \geq k$

Quiz: Which case should be considered when finding the time complexity of the following recurrence relation?

$$T(n) = 16T\left(\frac{n}{4}\right) + n$$

A: $\log_b a > k$

B: $\log_b a = k$

C: $\log_b a < k$

D: $\log_b a \geq k$



Correct Answer!

Since $\log_4 16 > 1$.

Was this helpful?

Yes

No